Arcora — Litepaper

Version 0.1 — 2026-05-13

> *Stablecoin checkout & settlement on Arc Network. Customer pays with what they have. Merchant settles in the stablecoin they want. One signature, one on-chain settlement, deterministic payout.*

Table of contents

---

1. Executive summary

Arcora is a stablecoin checkout protocol on Arc Network. A merchant invoices in their preferred stable (USDC, EURC, with more on the roadmap). A customer signs a single Permit2 EIP-712 message — no transaction, no gas. An Arcora-operated relayer pulls the pay-in token, runs Circle App Kit Swap to convert it, and calls the gateway's `settleInvoice` to deposit the payout into a custody escrow that the merchant claims after a 7-day refund window.

The intended end-state is **cross-chain**: customer pays from any source chain Arc App Kit Bridge supports, merchant still settles in their preferred Arc stable. That milestone is spec'd as v2.0 and not yet built. The current product (v1.1) ships Arc-side only — same UX shape, single-chain scope.

Current state (2026-05-13):

	Value
Stage	Pre-mainnet (Arc Network is testnet; we follow)
Live deploy	`arcorapay.xyz`
Contract	`ArcFXGatewayV11` at `0x07BAC123A682D24d3eC439ce454cA8AC64eAe3A3` (Arc testnet, chain 5042002)
Internal audit	Post-V10 audit 2026-05-12: **43 findings closed across 5 Critical, 18 High, 14 Medium, 4 contract test gaps, plus operability + a11y + DX cleanups** — see §6.2
External audits	Pulse-AI 2026-05-03 (all P1/P2 cleared) · Plan-7 PR #4 2026-05-05 (17 verified + 3 partial closed)
Contract test suite	77 Foundry tests, including fuzz invariants — all passing on V11 source
Token	None. The protocol does not issue or require a token.
Audience for this paper	Investors, integration partners, security reviewers

The rest of this document goes deep on architecture, security posture, and what's still open before mainnet.

2. The problem

Stablecoins moved roughly $5 trillion of value in 2025; USDC alone accounts for the bulk of that. For someone **holding** stables, the experience is fine — wallet to wallet on a single chain.

For a **merchant** accepting them, payment is still ugly:

- **Customer's chain ≠ merchant's chain.** A buyer holds USDC on Arbitrum; the merchant operates a euro-denominated business and wants EURC on Arc.
- **Customer's token ≠ merchant's token.** Even on a single chain, the customer wants to pay in whatever they have (USDT, PYUSD, regional pegs). The merchant wants one accounting currency.
- **Bridge + DEX + approval clicks.** Today's UX makes the customer plan a multi-step route. They learn about CCTP, slippage, gas tokens. Most don't bother.
- **Custodial off-ramps charge 1–2% and take days.** That's what merchants currently fall back to.

The alternative — "just accept any stablecoin and live with the FX risk" — pushes treasury complexity onto every merchant. That's not a product, that's a tax.

> *"I have USDC on Arbitrum. The merchant wants EURC on Arc. Five clicks and twenty minutes later I'm not sure if my money got there."*

Arcora collapses that flow into one signature.

---

3. What Arcora is

Arcora is a Stripe-like checkout for stablecoin payments. It runs as a non-custodial protocol on Arc Network with an off-chain orchestration layer that hides the FX, swap, and settlement plumbing behind one interface.

The single product bet:

> ***Customer pays from where they are, with whatever they have. Merchant receives their preferred stablecoin on Arc.***

Concretely, that means:

- **One signature on the customer side** — a Permit2 EIP-712 message authorising a specific transfer for a specific invoice. No native gas, no on-chain transaction the customer pays

for.

- **Deterministic payout on the merchant side** — the contract guarantees `>= amountOut` of the payout token lands in custody, or the customer is refunded. No "we'll see how the swap goes" UX.
- **A custody escrow window** — settled funds sit in the gateway for 7 days, claimable by the merchant after. During that window the merchant can issue refunds without merchant cooperation on the customer side. After it, the merchant calls `claim()` and the funds release.

There is no Arcora token. The protocol charges a basis-point fee on settlements (currently 30 bps, capped at 1000 bps in the constructor) that accrues to a withdrawable bucket; that is the protocol's only revenue surface today.

---

4. System architecture

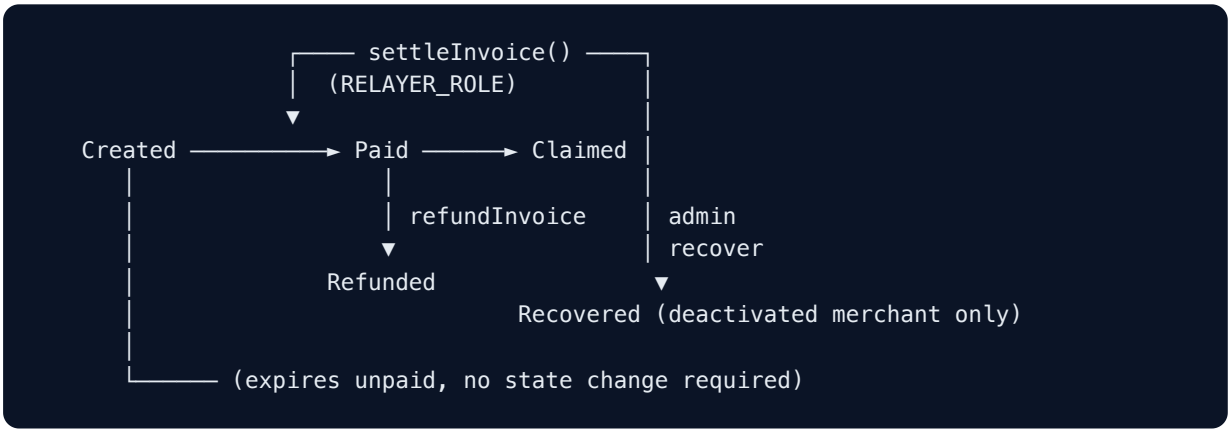
4.1 Component overview

```
│   ArcFXGatewayV10  0xc91e45ff…    (deprecating, refund window)    │
│   USDC             0x36000000…                                     │
│   EURC             0x89B50855…                                     │
│   Permit2          0x00000000…3aC78BA3                             │
│   App Kit Swap     RFQ network (Circle)                            │
└────────────────────────────────────────────────────────────────────┘
```

Three services share a single Postgres and watch a single chain. The Vercel app is the only customer- and merchant-facing surface; the three daemons (indexer, webhook, relayer) run on a single VPS and are operationally invisible to end users.

4.2 The custody-escrow gateway (V11)

The on-chain contract is intentionally small. It does not swap, it does not hold customer balances mid-flight — it is an invoice lifecycle machine plus a fee accumulator plus a refund safety net.



Key design choices, with the why:

- **`settleInvoice` deposits into `escrows[globalId]` rather than transferring straight to the merchant.** The merchant cannot front-run a refund. The customer is protected for the entire `REFUND_WINDOW` (7 days). After that the merchant `claim()`s, permissionlessly, in batches.
- **`InvoicePaid` event emits `excess = grossPayout − amountOut` in the `fee` field.** Rate-favourable swaps put the surplus into the protocol fee bucket; the event reflects what was actually accrued (V11 fix from the 2026-05-12 audit).
- **`settleInvoice` validates `payInToken == inv.payIn`.** A relayer error or hostile relayer cannot lie in the on-chain log about which stable the customer paid (V11 fix).
- **`recordPayerRefund` is the relayer's terminal-failure path.** When App Kit Swap returns less than `inv.amountOut`, the relayer doesn't settle — it returns the pay-in to the customer off-chain and emits this event so the indexer flips the invoice to `failed`. Customer made whole, merchant gets `invoice.failed` webhook.

- **`adminRecoverEscrow` covers abandoned merchants.** After `REFUND_WINDOW + ADMIN_RECOVERY_DELAY` (14 days total) and merchant deactivation, an admin can sweep escrows to a recovery wallet. Batch is atomic — one invalid id aborts the entire call.
- **All fund-moving paths carry `nonReentrant`.** Including `withdrawFees`, which was added in V11.

V11 fixed four behavioural gaps surfaced by the 2026-05-12 internal audit (§6.2). The Solidity source file name (`ArcFXGatewayV10.sol`) did not change because the ABI is byte-identical — lineage is tracked by deployed address.

4.3 The relayer

The relayer is the only off-chain component with custody-class authority. It holds the `RELAYER_ROLE` on the gateway and is the only address allowed to call `settleInvoice` and `recordPayerRefund`.

What it does per invoice:

1. Claim the next pending row in `relayer_queue` with `SELECT … FOR UPDATE SKIP LOCKED` — atomic, lease-bounded (8 minutes), reclaimable on crash.
2. Call `Permit2.permitWitnessTransferFrom` using the customer's signature. The witness binds the signature to `(invoiceId, relayer)` so it cannot be replayed against a different invoice or by a different relayer.
3. If `payInToken == payoutToken`, skip the swap. Otherwise call Circle App Kit Swap (`kit.swap`) and wait for the maker network to fill.
4. Verify the actually-received amount of payout token is `>= inv.amountOut`. If not, refund the customer's pay-in off-chain and emit `recordPayerRefund`. If yes, transfer `grossPayout` to the gateway and call `settleInvoice`.
5. Persist intermediate state at each on-chain step (`permit2_tx_hash`, `swap_tx_hash`, `swap_amount_out`, `settle_tx_hash`) so a daemon crash mid-flight is recoverable on next reclaim.

The relayer signs through a Vault-backed **`LocalAccount`** (§4.6) — no private key on disk in any `.env` file at any point.

Failure modes the relayer handles cleanly:

- Stuck-mempool `settleInvoice` tx: after `MAX_ATTEMPTS` reclaims with the same `settle_tx_hash`, the row is marked `failed` for operator triage so the operator can rebroadcast at higher gas or settle out-of-band.
- Stale Vault credentials: at boot the daemon AppRole-logs in once. Daily cron rotates the secret_id well before its 24h TTL. If the cron has not fired and the token has expired (audit prediction validated 2026-05-08 to 2026-05-13), the daemon crash-loops and Vault's

user-lockout protection trips — the operator recipe is documented and the rotation gap is now closed.

- App Kit Swap RFQ failure: surfaces as a swap revert; the row goes through the off-chain refund path above.

4.4 Indexer + webhooks

The indexer tails the chain and reconciles on-chain state with the database. Every event the gateway emits maps to one row update:

Event	What it does
`InvoiceCreated`	Inserts/idempotently confirms the invoice row. `gateway_address` reads from `log.address` so V10 + V11 are tagged distinctly during the dual-watch window.
`InvoicePaid`	Flips status to `paid`, fills in `paid_by`, `paid_tx`, `paid_at` (from block timestamp), `merchant_payout`, `protocol_fee`.
`SettlementContext`	Stitches `pay_in_actual` + `swap_tx_hash` into the same row (separate event for ABI cleanliness).
`EscrowCreated`	Sets `claimable_at` on the invoice row, partial index `idx_invoices_claimable_at` makes the "matured escrows" query fast.
`InvoiceRefunded`	Status → `refunded`. Indexer also accepts out-of-order arrivals (refunded before paid) to avoid permanent `created` lock.
`PayerRefunded`	Status → `failed` for the off-chain refund path.
`InvoiceClaimed`, `EscrowRecovered`, `MerchantReactivated`	Terminal lifecycle bookkeeping.

Cursor lives in `indexer_state.last_processed_block`; runs with `REORG_BUFFER=5` and a `MAX_RANGE=9000` per `eth_getLogs` chunk to fit Arc testnet's RPC cap. Catch-up replay is safe because every timestamp on the indexer side is now sourced from block time, not daemon wall-clock.

The webhook daemon consumes `webhook_attempts` rows the indexer emits. Each delivery:

- Signs the body with HMAC-SHA256 (`X-Arcora-Signature: sha256=<hex>`).
- Re-validates the destination URL through the SSRF guard at delivery time (not just at create time) — RFC1918 / loopback / link-local / CGN / IPv6-mapped IPv4 / cloud metadata

addresses are rejected.

- Backs off `2^attempts * 30` seconds on failure (capped at 24h), aligned with the relayer's reschedule formula so a single merchant outage doesn't churn the table.
- Marks `terminal_reason` permanently on a 4xx response so the row is never re-queued.

4.5 Compliance hooks

Phase 0 of the compliance design is live in production with the `NoopProvider` (testnet default — no calls made). A real provider can be activated entirely by env-var flip; the integration shape is in code and tested.

Two screening flows:

Flow	When	Behaviour
`merchant_payout`	On invoice creation (`/api/invoices`)	Screens the merchant's on-chain `payoutAddress` (snapshot read from the gateway, not the DB). `reject` → 403; `review` → 202 ticket; `allow` → continue.
`customer_pay`	On `/api/checkout/authorize`, before the Permit2 sign	Screens the customer's wallet. `reject` → button disabled with explanation; `review` → wallet-can't-be-used panel with ticket id; `allow` → button enabled.

Both flows cache results per address by provider TTL (sanctions hits live 7 years per regulation, other categories 13 months). The cache table is `compliance_screenings`. Failures default fail-closed for customer flow (better-safe-than-sorry on pay), fail-open for invoice creation (provider outage shouldn't block merchants from minting invoices).

KYB onboarding is spec'd separately (`ManualKybProvider` for testnet, `PersonaProvider` for post-revenue) and not yet wired into the merchant signup flow. That's mainnet-gated.

4.6 Key isolation (Vault)

The relayer's private key never lives in plaintext on disk anywhere. The pattern:

Properties:

- **Encrypted at rest** in Vault's KV-v2 (vault's own master key wraps the secret).
- **AppRole-gated** with daily `secret_id` rotation. `role_id` is long-lived; `secret_id` rotates well before its TTL.
- **Audit-logged** — every read of `secret/data/relayer-v10` writes a Vault audit-device entry.
- **Single fetch at boot** — V11 collapsed an earlier double-fetch (signer + adapter each fetched separately) into one round trip, removing a race window if the secret_id rotated mid-boot.
- **Rotation operator is least-privilege** — the cron's token holds policy `rotation-operator`, granting only `update auth/approle/role/relayer/secret-id`. It cannot read the key or do anything else. Replaced the root token used during initial deploy.

- **Loopback-only listener** — Vault accepts no remote connections; `tls_disable=1` is safe in that context. The trip-wires for re-enabling TLS are commented inline on the config so any future change that broadens the bind triggers a review.

Plan-11 (mainnet T-0) will move signing to a properly isolated HSM (AWS KMS Cloud HSM or a vetted Vault transit secp256k1 plugin) so the key never leaves the HSM boundary. Today's design is honest about the boundary it draws: the key lives in process memory after boot; if a process is compromised, the key is too. Mitigations are short-lived process restarts on env-file rewrite, no key rotation requirement mid-process, and the minimum-blast-radius design above.

---

5. Settlement flow, step by step

A complete USDC→EURC payment, end to end.

```
T+0:  Merchant creates invoice
      POST /api/invoices  (x-arcora-api-key: ak_live_…)
      → server compliance-screens merchant payoutAddress (cache hit if recent)
      → server calls createInvoiceFor() on V11 (server hot wallet)
      → invoice row inserted, gateway_address = V11
      → returns { invoiceId: 0x…, url: arcorapay.xyz/i/0x… }

T+5s: Customer opens checkout
      GET /i/<invoiceId>
      → page renders QuoteDisplayV8 + PayButtonV8 + Thirdweb ConnectButton

T+10s: Customer connects wallet
      → useComplianceGate triggers /api/checkout/authorize
      → server compliance-screens customer wallet
      → if allow: writes checkout_authorizations row with minAmountIn + statusToken
      → PayButton transitions from "Verifying wallet…" to "Pay with one signature"

T+15s: Customer signs Permit2
      → wallet shows EIP-712 typed-data with the witness components
        (invoiceId, relayer)
      → no transaction, no native gas
      → POST /api/checkout/submit  { permit2Signature, … }
      → server verifies witness hash matches, inserts relayer_queue row

T+20s: Relayer picks up the row (5s tick)
      → SELECT … FOR UPDATE SKIP LOCKED claims it, attempts = 1
      → calls Permit2.permitWitnessTransferFrom — pulls USDC into relayer
      → if same-token, skips swap; else calls kit.swap (App Kit RFQ)
      → maker fills, EURC arrives at the relayer address

T+25s: Settlement
      → relayer transfers grossPayout EURC to V11
      → calls gateway.settleInvoice(globalId, payer, USDC_addr,
                                    amountIn, grossPayout, swapTxHash)
      → V11 emits InvoicePaid + SettlementContext + EscrowCreated
      → V11 deposits grossPayout into escrows[globalId]
      → V11 status = Paid

T+27s: Indexer picks up the events (30s tick window)
      → UPDATE invoices SET status='paid', paid_at=block.timestamp, …
      → enqueues invoice.paid webhook

T+30s: Webhook daemon delivers
      → HMAC-signs body, POSTs to merchant's webhookUrl
      → SSRF guard re-validates URL at delivery
      → on 2xx response: row marked delivered. On 4xx: terminal_reason set.

T+0 + 7d: Merchant claims
      → calls gateway.claim([globalId, …])
      → escrow transfers to merchants[merchant].payoutAddress
      → V11 emits InvoiceClaimed
      → indexer flips status to 'claimed'
```

Refund (within window):

```
M  Merchant or refund-delegate calls gateway.refundInvoice(globalId)
   → V11 transfers escrow.amount back to the customer's payer wallet
   → escrow deleted, invoice status = Refunded
   → InvoiceRefunded event → indexer → invoice.refunded webhook
```

Failed swap (off-chain refund):

```
S  Relayer sees grossPayout < inv.amountOut
   → ERC20.transfer the USDC pulled-via-Permit2 back to the customer
   → gateway.recordPayerRefund(globalId, payer, USDC_addr, owedBack, reasonHash)
   → V11 status = Failed (no swap happened on-chain)
   → PayerRefunded event → indexer flips invoice to failed → invoice.failed webhook
```

---

6. Security posture

6.1 Threat model summary

Full document at `docs/audit/threat-model.md`. Headlines:

Actor	Trust	Reach
Customer (payer)	Untrusted	Permit2 signature scoped by witness to `(invoiceId, relayer)`; cannot replay across invoices or relayers
Merchant	Semi-trusted	API key (bcrypt-prefix-lookup), allowed-origins allowlist, on-chain `payoutAddress` snapshot-then-screen at invoice create
Relayer	Trusted within scope	RELAYER_ROLE only; cannot upgrade, withdraw fees, change roles, or pause. Hot wallet is a single key
Admin	Fully trusted	DEFAULT_ADMIN_ROLE; single EOA today, multisig pre-mainnet
Circle App Kit Swap	Trusted third party	Off-chain RFQ network; not on the contract attack surface

Adversary worth modelling for:

- Hostile or compromised relayer: cannot drain (no escrow withdrawal path with RELAYER_ROLE), cannot corrupt log (V11 validates `payInToken`), but can intentionally

settle worse-than-quote and bank the slippage to protocol fee — mitigated by `grossPayout < amountOut` revert path.

- Phishing attack on customer: Permit2 witness binds the signature to a specific invoice + relayer. A signature collected on a different domain or for a different invoice cannot authorise a transfer here.
- Hostile merchant: cannot reach customer funds (escrow held in contract, merchant can only claim after window), cannot redirect to arbitrary URLs (allowlist + SSRF guard on `successUrl` / `cancelUrl`), cannot run an unscreened payout wallet (on-chain `payoutAddress` is screened, not the identity wallet).
- DB compromise: API keys are bcrypt-prefixed (lookup uses prefix, hash verifies), webhook secrets are AES-256-GCM-encrypted at rest, SIWE nonces are atomic-consume. RPC + Vault are not reachable from the app's network surface.

6.2 Internal audit closure (2026-05-12)

A 5-parallel-subagent audit on 2026-05-12 surfaced 5 Critical, 18 High, 14 Medium, 4 contract test gaps, plus operability/a11y/DX cleanups. All closed on `plan-1-protocol` across 43 commits and validated end-to-end. Snapshot of the closure by phase:

Phase	Findings	Outcome
Phase 1 — Critical (5)	Drizzle migration journal sync · Shop `?engine=v9` hardcode · Demo-merchant API key in browser bundle · Vault policy ≠ runtime path · Rotation script env-path mismatch	All resolved + prod DB migration tracker backfilled with 19 correct rows + secrets layout aligned to the running config
Phase 2 — High (18)	`settleInvoice` payInToken validation · Logout CSRF + `PUBLIC_BASE_URL` fail-closed · `/merchant/escrows` query limits · API key header alias · Compliance gate `error` UX · Dashboard fetch error states · Merchant login WalletConnect · ClaimAll toast · Vault double-fetch · Stuck-mempool MAX_ATTEMPTS · Webhook backoff alignment · Indexer redundant SELECT · WC plugin URL + return guards · useCheckout dep array · SDK factory pattern	All resolved with per-finding commits and audit-id tags
Phase 3 — Medium + tests + cleanups (~25)	14 Medium findings (event emit math, address-truncation, BigInt arithmetic, UTF-8 byte slicing, dedicated user lines, …) · 4 contract test gaps closed with a new `V10AuditCoverage.t.sol` (payInToken mismatch, fee-math fuzz, `whenNotPaused` on `recordPayerRefund`, batch-atomicity on `adminRecoverEscrow`) · Migration 0018 (retention indexes) · A11y quick wins · Deployment manifest sync · `@deprecated` JSDoc	All resolved
Phase 4 — Post-audit follow-ups	Indexer `paid_at` + `refunded_at` block timestamps · Header alias test (canonical + legacy) · MerchantSidebar `chainId` derive · `relayer-smoke.yml` workflow_dispatch · **V11 deploy + soft cutover** · **Rotation operator de-privilege**	All resolved

The V11 deploy itself bakes in the four [next-version]-tagged contract fixes:

Audit ID	Behaviour
`#6`	`settleInvoice` reverts `InvalidPayInToken()` when caller-supplied `payInToken ≠ inv.payIn`
`#25`	`InvoicePaid` emits `excess = grossPayout − inv.amountOut` as the `fee` field instead of hardcoded 0
`#26`	`DeployV10` logs a loud WARN when `SUPPORTED_TOKENS` is set but deployer ≠ owner (`setTokenSupport` would silently no-op)
`#27`	`withdrawFees` carries `nonReentrant` for consistency with every other fund-moving path

A production outage was discovered and resolved during the follow-up work. Audit finding `#5` predicted that the rotation script's mismatched env path would lead to crash-looping after the 24h `secret_id` TTL expired. That prediction landed in production: the relayer was crash-looping for 5 days (restart counter ~40080) until the AppRole was unlocked, a fresh secret_id minted, and the service restarted. Daily rotation cron is now installed and uses a least-privilege operator token (not root). Recovery recipe captured in memory.

6.3 External audit history

When	Engagement	Scope	Outcome
2026-05-03	Pulse-AI external audit	V9 gateway + adjacent off-chain surfaces	All 6 P1/P2 findings closed same day (submit hardening, SIWE domain+chain bind, webhook SSRF guard, refund-source binding, lease-reclaim, workspace layout)
2026-05-05	Plan-7 PR #4 (subagent-driven external review)	Full repo	17 verified findings + 3 partial closed across 28 commits; 4 V10-deferred items (custody, fee bound, reactivate, nonReentrant) all closed in the V10 deploy itself

Both audits' findings, evidence, and closure rationale are tracked in `docs/audit/`. **The internal 2026-05-12 audit was scoped to "what those audits missed or V10 introduced" — anything they already covered was out of scope.**

External engagement for mainnet T-0 (Spearbit / Cantina / Sherlock RFP) is gated on "first paying merchant OR funding round closes — whichever comes first" per the project's mainnet bar policy.

6.4 Operational hardening

Surface	Practice
API keys (server-side at issue)	bcrypt with searchable prefix; lookup hits the prefix index, hash verifies. Never stored or logged plaintext.
Server wallet (pays invoice-creation gas)	AES-256-GCM encrypted at rest, 12-byte IV + auth tag, 32-byte master key from env
Relayer key	Vault KV-v2 + AppRole; encrypted at rest, daily `secret_id` rotation via least-privilege operator token (§4.6)
Webhook secrets	AES-256-GCM at rest; revealed once at rotation, never re-fetchable
SIWE	Domain bind + chain bind (derived from `arcTestnet.id`, not magic number) + atomic nonce consume (single UPDATE with WHERE clauses)
Redirect URLs (`successUrl` / `cancelUrl`)	Three-layer guard: merchant allowlist + server-side SSRF re-resolve + client-side `safeClientRedirect`
Rate limit	Postgres-backed counters; `/api/checkout/quote` is 30 req / 60s per IP to protect the KIT_KEY quota
Webhook delivery	HMAC-SHA256 signed body, SSRF guard at delivery time (re-resolves DNS), 4xx terminal, aligned backoff
Compliance retention	7y for sanctions hits, 13mo for other categories. Indexed on `expires_at` for cron prune (audit-driven addition)
Audit logs	Vault audit device on enable (off by default to keep dev clean); ops logs to systemd journal with `SyslogIdentifier` per service
Service supervision	systemd `Restart=always`, `RestartSec=5`, `StartLimitBurst=5` per 60s window — crash loops can't fill the journal

7. Deploy state & address book

Arc Network testnet (chain id 5042002, RPC `https://rpc.testnet.arc.network`, explorer `https://testnet.arcscan.app`):

Component	Address	Notes
`ArcFXGatewayV11`	`0x07BAC123A682D24d3eC439ce454cA8AC64eAe3A3`	**Current.** Deployed 2026-05-13. Tx `0x4a59990e…`. Audit-fix bytecode.
`ArcFXGatewayV10`	`0xc91e45ffe945c0e6e2c0f8262a35477e20a5f154`	Deprecating. Lives on-chain through ~2026-05-27 for in-flight refund + recovery windows.
`Permit2`	`0x000000000022D473030F116dDEE9F6B43aC78BA3`	Uniswap canonical, same on every chain
USDC	`0x3600000000000000000000000000000000000000`	6 decimals
EURC	`0x89B50855Aa3bE2F677cD6303Cec089B5F319D72a`	6 decimals
Gateway owner / deployer	`0xe8E5AAa3d8c705A07de02aADF98CE31F20A5754b`	Pays oracle keepalive and deploy gas
Relayer (V11)	`0x29EcFedDF31E4dA4a62b89bADe35b224cE144DAE`	Vault-derived; same key as V10
Server hot wallet	`0x74BB69F48d0dAddB17679534fDa1142c3ab66333`	Pays invoice-creation gas (AES-encrypted at rest)

V11 constructor parameters (1:1 with V10):

Param	Value
`protocolFeeBps`	30
`REFUND_WINDOW`	604800 (7 days)
`ADMIN_RECOVERY_DELAY`	604800 (7 days)
Whitelisted tokens	USDC + EURC (set in deploy script)

Hosting:

- App: Vercel (Next.js App Router, Turbopack), `arcorapay.xyz`
- DB: Neon Postgres (EU central), pgbouncer + non-pooling DSN

- Daemons: single Ubuntu 24.04 VPS (`relayer.service` , `indexer.service` , `webhooks.service`)
- Secrets: HashiCorp Vault on same VPS, loopback listener, file backend

Test surface:

- Foundry: 77 contract tests including a fuzz invariant on fee math, all passing on V11 source
- Vitest: server-side test suites for app + SDK; 17 SDK tests, dozens of route-handler unit + integration tests
- Lint + typecheck: clean on `plan-1-protocol`

---

8. Developer surface

SDK (`@arcora/sdk` v1.1):

```
import { Arcora } from "@arcora/sdk";

const arcora = new Arcora({ apiKey: "ak_live_…", environment: "testnet" });

const inv = await arcora.createInvoice({
  amountUsdc: 49.99,
  payInToken: "EURC",
  successUrl: "https://yoursite.com/order/123/success",
});

arcora.openCheckout(inv);
```

ESM + CJS + IIFE (`<script>`) builds. Strict TypeScript surface (no `any` on public). Tagged `@deprecated` on the legacy singleton (`Arcora.init`) — instance API is the canonical path forward.

React SDK (`@arcora/sdk-react`):

```tsx
import { useCheckout } from "@arcora/sdk-react";

function PayButton() {
  const { checkout, loading, error } = useCheckout({ apiKey, environment: "testnet"
  return (
    <button
      onClick={() => checkout({ amountUsdc: 4.5, payInToken: "EURC", successUrl: "…
      disabled={loading}
    >
      Pay €4.50
    </button>
  );
}
```

HTTP API (server-to-server):

Endpoint	Method	Auth	Purpose
`/api/invoices`	POST	`X-Arcora-Api-Key`	Create invoice
`/api/invoices/[id]`	GET	`X-Arcora-Api-Key` (canonical) or `x-api-key` (legacy)	Read invoice; metadata exposed only to owning merchant
`/api/merchant/escrows`	GET	iron-session (dashboard)	V10/V11 escrow buckets: pending / matured / claimed
`/api/merchant/treasury`	GET	iron-session	Per-stable rollups + activity feed

Webhooks (delivered to merchant-configured URL):

```
POST https://merchant.example.com/webhooks/arcora
Content-Type: application/json
X-Arcora-Signature: sha256=<hex>
User-Agent: arcora-webhook/1.0

{ "type": "invoice.paid",
  "invoice_id": "0x…",
  "payer": "0x…",
  "tx_hash": "0x…",
  "occurred_at": "2026-05-13T12:06:56Z" }
```

Signature is computed over the raw body with the per-merchant webhook secret. The verification helper uses `hash_equals` (PHP) or `crypto.timingSafeEqual` (Node) in our

reference plugins.

Plugin: `arcora-woocommerce` ships in-tree. Shopify is planned (v1.x #6).

---

9. Roadmap

v1.0 — Arc-only checkout — ✅ shipped 2026-04 → 2026-05

USDC and EURC, Permit2-based settlement, hosted checkout, merchant dashboard, refunds, treasury reporting. SDK + React SDK + WooCommerce plugin published.

v1.1 — Custody escrow + audit closure — ✅ shipped 2026-05-13 (V11)

Custody-escrow gateway, refund safety net, admin recovery for abandoned merchants, compliance gate (Phase 0), Vault KV key isolation, two external audits closed, internal audit closed. **This is the current production release.**

v1.x — In-flight enhancements (running in parallel)

Item	Status	Notes
Multi-stable (USDT, PYUSD, DAI, USDe)	In flight	Mainnet-only; testnet App Kit Swap is USDC/EURC only
Shopify plugin	Spec'd	After WC stabilises
Plugins for Stripe-like JS / React / WC / Shopify	Partial (JS + React + WC done, Shopify pending)	

v2.0 — Cross-chain USDC source — spec'd

Customer pays USDC from any CCTP-supported EVM chain (Ethereum, Arbitrum, Optimism, Base, Polygon, Avalanche, Linea, Codex). Merchant still settles in their chosen Arc stable. Routes through **Arc App Kit Bridge** (the supported primitive) rather than raw CCTP. New `payment_routes` table tracks the multi-step state machine. No new gateway contract — current V11 absorbs the settlement leg.

Critical pre-flight items (from the v2.0 spec):

- Verify CCTP Fast Transfer (or App Kit Bridge equivalent) latency on each source chain before claiming sub-minute UX
- Refund-from-each-intermediate-state path for the bridge sequence

- Arc-side stable liquidity per merchant payout token

v2.1 — Source-side DEX aggregator

Customer pays in any token on the source chain (native ETH, any ERC20), not just USDC. Odos / 1inch / 0x / Paraswap router on each source chain, reverse route engine for `target → Arc stable → CCTP → source token`. Two or three signatures depending on Permit2 availability on source.

v2.2 — Non-EVM sources

Solana, Sui, etc. Separate SDK + wallet stack per family; ~month of integration each after v2.1 is stable.

v3.0 — Intent / solver model

True one-click "Stripe-like" UX. User signs one intent; Arcora's solver executes the full route. Compare ERC-7683 / Across / DeBridge-Liquid. Multi-month research effort; not started until v2.0 + v2.1 are stable.

---

10. Mainnet pre-flight

A single ordered list of what crosses from "testnet good" to "mainnet ready". None of this is hypothetical work; each item is concrete and has a defined trigger.

#	Item	State	Trigger
1	Arc Network mainnet launch	Out of our control	Arc team timeline
2	External audit (Spearbit / Cantina / Sherlock RFP)	Layers 1+2 done (Slither + Mythril in CI, threat model, NatSpec, coverage gate, docs)	First paying merchant OR funding round close
3	Multisig admin migration	Single EOA today	T-0
4	KYB onboarding	`ManualKybProvider` + `PersonaProvider` spec'd; integration not yet wired	Pre-revenue cutover
5	Vault TLS (cert + listener config)	`tls_disable=1` safe today (loopback-only); trip-wires documented	T-0
6	Dedicated `arcora-ops` Unix user	Service units have commented `User=` lines; needs file moves out of `/root` + chown + reconcile Vault AppRole perms	T-0
7	HSM signer migration	Today: in-process viem signer; App Kit still requires raw key → needs upstream or adapter work	T-0
8	Rotation operator token renewal cron	Token TTL capped to 32d on this Vault; renewable	Before 32d expiry
9	Compliance provider activation	NoopProvider live; Elliptic + TRM adapters scaffolded behind env flag	KYB go-live
10	Real Chainlink price feeds	MockChainlinkFeed today	T-0
11	Domain canonicalisation	`arcorapay.xyz` live; `arcorapay.com` desired	Branding decision
12	Plan-7 Layer 3 — Immunefi	Deferred	Mainnet T-0 with first paying merchant

Inside scope items 5–8 are reversible single-host changes. Items 1–4, 9, 10 are coordination-and-process items that don't strictly block the protocol but block a "yes you may take real money" answer.

11. Glossary

- **Permit2**: Uniswap's canonical EIP-712 transfer authorisation contract at `0x000000000022D473030F116dDEE9F6B43aC78BA3`. Lets a customer authorise a single transfer without an on-chain `approve` transaction; with a **witness**, the authorisation is scoped to a specific application context (here: `(invoiceId, relayer)`).
- **App Kit Swap**: Circle's RFQ-based stablecoin swap on Arc. Maker network fills swap requests; merchant integration is via the `@circle-fin/app-kit` SDK. Used by the relayer for the FX leg.
- **Custody escrow**: The pattern where settled funds sit in the contract (not the merchant's wallet) until a refund window closes, after which the merchant permissionlessly claims. Arcora V10/V11 use a 7-day refund window + 7-day admin-recovery delay.
- **SIWE**: Sign-In With Ethereum (EIP-4361). Used for merchant dashboard auth; we bind by domain + chain id + atomic nonce.
- **AppRole**: Vault's machine-auth method. A `role_id` (long-lived, identifies the role) + a `secret_id` (rotatable, time-limited) are exchanged for a Vault token. Used to fetch the relayer key at boot.
- **KV-v2**: Vault's versioned key-value secrets engine. Reads use the path `secret/data/<name>`; writes use `secret/data/<name>` with a `data:` wrapper.
- **Iron Session**: The cookie-based session library used for the merchant dashboard. Encrypted, signed, 7-day TTL.
- **Soft cutover**: Deploying a new contract version while the previous one stays callable for in-flight commitments. Used for V10 → V11: new invoices route to V11; existing V10 escrows finish their 7-day refund window before V10 retires.

Document history

Version	Date	Author	Notes
0.1	2026-05-13	Arcora	Initial litepaper. Reflects V11 LIVE state and 2026-05-12 audit closure.

This document captures live state at the date stamped above. Contract addresses, ops topology, and roadmap items change over time — `packages/contracts/deployments/arc-`

`testnet.json` is the canonical record for on-chain state; the `MEMORY.md` index in the repo root tracks operational follow-ups.